

Module 6 Lab Session 1: The Predictable Contract

Module	M6 RESTful Web APIs (TMS contract)
Session	1 of 3
Exercises	1 (First REST Controller), 2 (DTOs and Validation), 3 (Status Codes and Business Rule Errors)

Welcome to the API Contract Sprint

In M5 you wired the TMS to a real PostgreSQL database. EF Core knows how to materialise Course, Student, and Enrollment rows; SQL logging shows you the exact SQL behind every LINQ call. The plumbing works. Now imagine handing what you have to an Angular team: there is no CoursesController, the only routes that exist are M4's EnrollmentsController proof-of-life, and even if there were a controller, it would be returning EF entities with circular Enrollments navigation properties that crash the JSON serialiser.

This session you build the first slice of the API contract that the Angular team in M8 and any future integration partner will actually consume. Three exercises, one controller (CoursesController), one supporting controller (EnrollmentsController), and one mental model: **for a single resource, the full contract should exist before scale concerns enter.**

By the end of Session 1 your TMS API answers, for Course and Enrollment:

- Resource-named routes (/api/courses, /api/courses/{id}/enrollments) instead of verb-in-URL routes.
- DTO firewall between EF entities and the wire no IsDeleted, no navigation property leak.
- Data Annotations validation that returns 400 Bad Request with ValidationProblemDetails automatically.
- A 409 Conflict with ProblemDetails when an admin tries to create a duplicate course code or enrol into a full course the two business-rule failures this module cares about most.

Pagination, HATEOAS, and Scalar metadata are deliberately not in this session. The full contract for **one** resource has to land first.

Before You Begin Align the M5 Codebase to M6's Course Shape

M5's Course entity already has Id, Code, Title, and Capacity (with Code configured as a unique index in Module 5 Session 2). For Module 6, we need to perform one column rename to align with standard TMS naming conventions: rename Capacity to MaxCapacity. We will also confirm the unique index configuration for Code is in place.

1. Update the entity

In your M5 entities, edit Course.cs:

```
public class Course
{
    public int Id { get; set; }
    public required string Code { get; set; }
    public required string Title { get; set; }
    public int MaxCapacity { get; set; }
    public ICollection<Enrollment> Enrollments { get; set; } = [];
}
```

Confirm that CourseConfiguration has the rules for both properties configured:

```
public void Configure(EntityTypeBuilder<Course> builder)
{
    builder.HasKey(c => c.Id);
    builder.Property(c => c.Code).IsRequired().HasMaxLength(10);
    builder.Property(c => c.Title).IsRequired().HasMaxLength(200);
    builder.HasIndex(c => c.Code).IsUnique();
    builder.HasMany(c => c.Enrollments).WithOne(e => e.Course).HasForeignKey(e => e.CourseId);
}
```

The unique index on Code is what makes the duplicate check in Exercise 3 a real business rule and not a courtesy the database itself rejects the second insert.

2. Add and apply the migration

From your solution root:

```
dotnet ef migrations add RenameCapacityToMaxCapacity
```

Open the generated migration file. You should see a RenameColumn for Capacity → MaxCapacity (or an AddColumn + DropColumn pair if EF could not detect the rename). Since Code and its unique index are already defined from Module 5, they will not generate any new database changes. If you see a DropTable of Courses

followed by CreateTable, **stop** your model has drifted further than this exercise expects. Roll back the model change, fix the entity, and try the migration again.

```
dotnet ef database update
```

Verify in pgAdmin: Courses table should now show Id, Code, Title, MaxCapacity.

3. Confirm the M4/M5 services and middleware are still wired

Open Program.cs in your API project and confirm (without changing anything yet) that the M4 baseline is intact:

```
builder.Services.AddProblemDetails();
builder.Services.AddOpenApi();
builder.Services.AddDbContext<TmsDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("TmsDatabase")));
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.UseExceptionHandler();
app.UseStatusCodePages();
```

```
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}
```

```
app.MapControllers();
app.Run();
```

If AddProblemDetails, UseExceptionHandler, UseStatusCodePages, or MapControllers are missing, fix those before you continue. Exercises 2 and 3 rely on the framework returning ValidationProblemDetails automatically and on unhandled exceptions becoming 500 ProblemDetails instead of HTML stack traces.

Checkpoint: dotnet build succeeds, the API starts on https://localhost:<your-port>, and https://localhost:<port>/scalar/v1 shows whatever endpoints M4 and M5 left behind. Course rows can be inserted with the new shape via psql or pgAdmin.

Exercise 1: Your First REST Controller (LO 6.1)

The TMS needs GET /api/courses/{id} to fetch a single course and POST /api/courses to create one. Both endpoints must return the correct REST status codes, accept a CancellationToken, and live behind a thin controller that does no business logic.

Step 1 Define the read-side service contract

Real .NET teams do not let controllers reach into DbContext directly. They put a service between the controller and persistence so business rules and side effects can be tested in isolation, and so the controller stays focused on HTTP concerns. You will build both halves.

Create Services/ICourseService.cs in your API project:

```
using Tms.Api.Entities;

namespace Tms.Api.Services;

public interface ICourseService
{
    Task<Course?> GetByIdAsync(int id, CancellationToken ct);

    Task<Course> CreateAsync(Course course, CancellationToken ct);
}
```

Create Services/CourseService.cs:

```
using Microsoft.EntityFrameworkCore;
using Tms.Api.Data;
using Tms.Api.Entities;

namespace Tms.Api.Services;

public class CourseService(TmsDbContext context, ILogger<CourseService>
    logger) : ICourseService
{
    public async Task<Course?> GetByIdAsync(int id, CancellationToken c
        t)
    {
        // TODO 1: Use context.Courses.AsNoTracking()
        //           and return FirstOrDefaultAsync(c => c.Id == id, ct).
        throw new NotImplementedException();
    }

    public async Task<Course> CreateAsync(Course course, CancellationTo
```

```

    ken ct)
    {
        // TODO 2: Add course to context.Courses,
        //           SaveChangesAsync(ct), log info, and return the creat
        ed course.
        throw new NotImplementedException();
    }
}

```

Notice three habits already in place: primary-constructor DI for the dependencies, `ILogger<CourseService>` (logging belongs at the service boundary, not in controllers), and `CancellationToken` on every async method.

Step 2 Register the service in Program.cs

A service is useless until DI knows about it. Above `var app = builder.Build();`, register the service **scoped** same lifetime as `TmsDbContext`, fresh per request:

```
builder.Services.AddScoped<ICourseService, CourseService>();
```

Why **scoped**, not **singleton** or **transient**? `CourseService` depends on `TmsDbContext`, which is **scoped**. Registering the service **singleton** would capture the `DbContext` instance forever and crash the second request. Registering it **transient** would create a new `CourseService` every time you ask for one in the same request pointless allocation. **Scoped** matches the unit-of-work boundary EF Core expects.

Step 3 Create the controller

Create `Controllers/CoursesController.cs`:

```

using Microsoft.AspNetCore.Mvc;
using Tms.Api.Entities;
using Tms.Api.Services;

namespace Tms.Api.Controllers;

[ApiController]
[Route("api/courses")]
public class CoursesController(ICourseService courseService) : ControllerBase
{
    [HttpGet("{id:int}", Name = nameof(GetCourseById))]
    public async Task<IActionResult> GetCourseById(int id, CancellationToken ct)
    {
        // TODO 3: Call courseService.GetByIdAsync(id, ct).
        //           Return Ok(course) when the result is not null.
    }
}

```

```

        //          Return NotFound() when the result is null.
        throw new NotImplementedException();
    }

    [HttpPost]
    public async Task<IActionResult> CreateCourse(Course course, CancellationToken ct)
    {
        // TODO 4: Call courseService.CreateAsync(course, ct).
        //          Return CreatedAtAction(nameof(GetCourseById), new {
id = result.Id }, result).
        //          CreatedAtAction sets the Location header automatically.
        throw new NotImplementedException();
    }
}

```

Three details that matter in production code:

- [Route("api/courses")] plural noun, no verbs, exactly the M6 Topic 1 rule.
- {id:int} route constraint bad ID like /api/courses/abc returns 404 immediately at the routing layer, never hits your action. Production APIs use route constraints to fail fast.
- Name = nameof(GetCourseById) names the route. Exercise 5 uses this name with LinkGenerator so the HATEOAS self link survives a future route refactor. Adding the name now costs nothing; adding it later means visiting every [HttpGet] in the codebase.

Step 4 Verify

1. Run the API. The build must succeed with the NotImplementedException placeholders intact (the methods compile they just throw at runtime).
2. Open <https://localhost:<port>/scalar/v1>. You should see GET /api/courses/{id} and POST /api/courses listed under whatever group Scalar puts un-tagged endpoints.
3. Once you fill in TODO 3 and TODO 4 (using the hints no separate worked solution lives in this handout), exercise the endpoints:

Step	Action	Expected status	Expected body / header
1	POST /api/courses body { "code": "CSE-101", "title": "Web Development Fundamentals", "maxCapacity": 30 }	201 Created	Location: /api/courses/{newId} header; body contains the created course entity
2	GET /api/courses/{newId} (use the ID from step 1)	200 OK	{ "id": ..., "code": "CSE- 101", "title": "Web Development Fundamentals", "maxCapacity": 30, "enrollments": [] }
3	GET /api/courses/9999 (assuming no course with that ID)	404 Not Found	Empty body or framework ProblemDetails shape

If step 1 returns 200 OK instead of 201 Created, you used `return Ok(...)` instead of `return CreatedAtAction(...)`. If step 1 returns 201 but with no Location header, the `nameof(GetCourseById)` reference is wrong confirm the action method name matches.

Troubleshooting

Symptom	Likely cause	Fix
404 on every /api/courses route	<code>app.MapControllers()</code> missing in <code>Program.cs</code>	Add it after <code>UseStatusCodePages()</code> , before <code>app.Run()</code>
InvalidOperationException on first POST	<code>ICourseService</code> not registered	Add <code>builder.Services.AddScoped<ICourseService, CourseService>();</code> in <code>Program.cs</code>
Scalar shows nothing under /scalar/v1	<code>AddOpenApi()</code> / <code>MapOpenApi()</code> / <code>MapScalarApiReference()</code> missing	Re-check the M4 wiring listed in Before You Begin
POST returns 200 OK instead of 201	Used <code>Ok(result)</code> instead of <code>CreatedAtAction(...)</code>	Replace with the <code>CreatedAtAction(nameof(GetCourseById), new { id = result.Id }, result)</code> pattern
404 on the GET that is supposed	<code>[HttpGet("{id}")]</code> does not match the	Both must use the same action name and the same route name

Symptom	Likely cause	Fix
to return 200	[HttpGet("{id:int}", Name = ...)] you used in CreatedAtAction	

Exercise 2: DTOs and Input Validation (LO 6.2)

Two issues hit during a peer review of Exercise 1. First, the Course entity has an Enrollments navigation property returning that to the wire produces a circular reference crash the moment a course has even one enrollment. Second, an admin clicked through Scalar and successfully created a course with an empty title the API accepted whatever JSON arrived. Fix both with a DTO firewall and Data Annotations.

Step 1 Define the response DTO

Create Dtos/CourseResponseDto.cs:

```
namespace Tms.Api.Dtos;

public record CourseResponseDto(
    int Id,
    string Code,
    string Title,
    int MaxCapacity,
    int EnrollmentCount);
```

A record is the right shape here DTOs are immutable value containers, and the positional syntax keeps the type to a single line. EnrollmentCount is computed at the query boundary, so the wire format never includes a navigation property.

Step 2 Define the request DTO with validation

Create Dtos/CreateCourseRequest.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Tms.Api.Dtos;

public record CreateCourseRequest
{
    [Required, RegularExpression(@"^[A-Z]{3}-\d{3}$",
        ErrorMessage = "Code must follow the pattern XXX-000 (e.g., CSE-101).")]
    public required string Code { get; init; }
```

```

[Required, MaxLength(200)]
public required string Title { get; init; }

[Range(1, 200)]
public int MaxCapacity { get; init; }
}

```

The required modifier and init accessors are the modern C# pattern: the compiler refuses to construct a `CreateCourseRequest` without `Code` and `Title`, but neither field can be reassigned after binding. The `[ApiController]` attribute on the controller wires up automatic validation against these annotations you do not### Step 3 Update the service contract and projection

Now that you have defined the DTOs, update the service interface `Services/ICourseService.cs` to return and accept them:

```

using Tms.Api.Dtos;

namespace Tms.Api.Services;

public interface ICourseService
{
    Task<CourseResponseDto?> GetByIdAsync(int id, CancellationToken ct);

    Task<CourseResponseDto> CreateAsync(CreateCourseRequest request, Ca
ncellationToken ct);
}

```

Open `Services/CourseService.cs`. Update its signature and replace its method bodies to map from/to these DTOs:

```

public Task<CourseResponseDto?> GetByIdAsync(int id, CancellationToken
ct) =>
    context.Courses
        .AsNoTracking()
        .Where(c => c.Id == id)
        .Select(c => new CourseResponseDto(
            c.Id, c.Code, c.Title, c.MaxCapacity, c.Enrollments.Count))
        .FirstOrDefaultAsync(ct);

```

Two production habits visible here: `AsNoTracking()` (read paths never need change tracking saves CPU and memory in EF) and `Select(...)` projection at the query layer (EF translates `c.Enrollments.Count` into a SQL `COUNT(*)` subquery; you do **not** load every enrollment row into memory just to compute a number).

For the create path, the cleanest pattern is to insert the entity, save, then re-query through GetByIdAsync so the response uses the same projection:

```
public async Task<CourseResponseDto> CreateAsync(CreateCourseRequest request, CancellationToken ct)
{
    var course = new Course
    {
        Code = request.Code,
        Title = request.Title,
        MaxCapacity = request.MaxCapacity
    };
    context.Courses.Add(course);
    await context.SaveChangesAsync(ct);
    logger.LogInformation("Created course {CourseId} ({Code})", course.Id, course.Code);
    return (await GetByIdAsync(course.Id, ct))!;
}
```

The null! forgiveness on the return value is safe here because we just inserted the row and saved it cannot be missing on the next read. The log line is the kind of breadcrumb you want in production: a single info log per write, never a log per read.

Step 4 Refactor the controller to use DTOs

Open Controllers/CoursesController.cs and refactor it to accept and return the DTOs instead of raw entities. Make sure to import using Tms.Api.Dtos;

```
using Microsoft.AspNetCore.Mvc;
using Tms.Api.Dtos;
using Tms.Api.Services;

namespace Tms.Api.Controllers;

[ApiController]
[Route("api/courses")]
public class CoursesController(ICourseService courseService) : ControllerBase
{
    [HttpGet("{id:int}", Name = nameof(GetCourseById))]
    public async Task<IActionResult> GetCourseById(int id, CancellationToken ct)
    {
        var course = await courseService.GetByIdAsync(id, ct);
        return course is not null ? Ok(course) : NotFound();
    }
}
```

```

[HttpPost]
public async Task<IActionResult> CreateCourse(CreateCourseRequest request, CancellationToken ct)
{
    var result = await courseService.CreateAsync(request, ct);
    return CreatedAtAction(nameof(GetCourseById), new { id = result.Id }, result);
}

```

By binding to `CreateCourseRequest`, the `[ApiController]` attribute automatically triggers model validation against the Data Annotations you set in Step 2.

Step 5 Verify

Step	Action	Expected status	Expected body
1	POST /api/courses body { }	400 Bad Request	ValidationProblemDetails JSON with errors.Code, errors.Title, errors.MaxCapacity field arrays
2	POST /api/courses body { "code": "abc", "title": "x", "maxCapacity": 9999 }	400 Bad Request	errors.Code mentions the XXX-000 pattern, errors.MaxCapacity mentions the 1–200 range
3	POST /api/courses body { "code": "CSE-102", "title": "TypeScript Essentials", "maxCapacity": 25 }	201 Created	{ "id": ..., "code": "CSE-102", "title": "TypeScript Essentials", "maxCapacity": 25, "enrollmentCount": 0 }
4	GET /api/courses/{id} from step 3	200 OK	The same DTO shape and crucially no enrollments array, no isDeleted, no navigation property

The minimum bar to clear here is step 4. If your response includes any property that is not on `CourseResponseDto`, your service is still returning a `Course` entity somewhere find the path and project before returning.

Troubleshooting

Symptom	Likely cause	Fix
POST {} returns 200 OK instead of 400	Missing [ApiController] on the controller	Add [ApiController] it enables automatic model validation
Response body contains enrollments array JsonException: A possible object cycle was detected	Returning the Course entity directly somewhere Same as above entity leak	Confirm every return path goes through CourseResponseDto projection DTO projection eliminates the cycle; never serialise navigation properties to clients
Regex rejects cse-101	Wrong-case input	The regex <code>^[A-Z]{3}-\d{3}\$</code> is case-sensitive on purpose; uppercase is the contract
Validation passes when MaxCapacity is 0	[Range(1, 200)] not applied	Confirm the attribute is on CreateCourseRequest.MaxCapacity and DTO is the bound type

Exercise 3: Status Codes and Business Rule Errors (LO 6.3)

The Exercise 1 + 2 path handles input validation cleanly but breaks on two business rule failures. First, posting a duplicate course code returns 500 Internal Server Error because the unique-index INSERT from your M5 migration crashes inside SaveChangesAsync. Second, when the M4 in-memory EnrollmentService is replaced with a database-backed enrolment endpoint, enrolling into a full course should return 409 Conflict instead it crashes the same way. You fix both by **checking the rule before the database does**.

Step 1 Add the duplicate-code check to the service

Extend ICourseService.cs:

```
public interface ICourseService
{
    Task<CourseResponseDto?> GetByIdAsync(int id, CancellationToken ct);
    Task<CourseResponseDto> CreateAsync(CreateCourseRequest request, CancellationToken ct);

    Task<bool> CodeExistsAsync(string code, CancellationToken ct);
}
```

In CourseService.cs:

```
public Task<bool> CodeExistsAsync(string code, CancellationToken ct) =>
    context.Courses.AsNoTracking().AnyAsync(c => c.Code == code, ct);
```

AnyAsync is the right call EF translates it to SELECT EXISTS (SELECT 1 ... LIMIT 1), which stops at the first row.

Step 2 Have the controller return 409 Conflict

In CoursesController.cs, replace TODO 4 (CreateCourse) with the pre-check pattern:

```
[HttpPost]
public async Task<IActionResult> CreateCourse(CreateCourseRequest request, CancellationToken ct)
{
    // TODO 1: Call courseService.CodeExistsAsync(request.Code, ct).
    //           If it returns true, return Conflict(new ProblemDetails
{ ... }) with:
    //           Title = "Course code already exists"
    //           Detail = $"A course with code '{request.Code}' is already registered."
    //           Status = StatusCodes.Status409Conflict
    //           You do not need a try/catch the framework's ProblemDetails middleware handles unhandled exceptions.

    var result = await courseService.CreateAsync(request, ct);
    return CreatedAtAction(nameof(GetCourseById), new { id = result.Id }, result);
}
```

Bad Practices: returning 200 OK for an error condition (Status Smokescreen) and a 409 without a valid ProblemDetails body (Missing Contract). Both are observable just by reading the response in Scalar.

Step 3 Add the enrolment side

Create the request DTO at Dtos/EnrollStudentRequest.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Tms.Api.Dtos;

public record EnrollStudentRequest
{
    [Range(1, int.MaxValue, ErrorMessage = "StudentId must be a positive integer.")]
    int StudentId { get; }
}
```

```

        public required int StudentId { get; init; }
    }

```

Create the response DTO at Dtos/EnrollmentResponseDto.cs:

```

namespace Tms.Api.Dtos;

public record EnrollmentResponseDto(
    int Id,
    int CourseId,
    int StudentId,
    DateTime EnrolledAt);

```

Create Services/IEnrollmentService.cs:

```

namespace Tms.Api.Services;

public interface IEnrollmentService
{
    Task<EnrollmentResponseDto?> GetByIdAsync(int courseId, int id, Cancellation token ct);

    Task<EnrollmentResponseDto> CreateAsync(int courseId, EnrollmentStudentRequest request, Cancellation token ct);
}

```

Create Services/EnrollmentService.cs. Two of the three method bodies are scaffolding (constructor + read path); the **graded** business decision the order of the look-ups, which 4xx code to choose at which gate is yours:

```

public class EnrollmentService(TmsDbContext context, ILogger<EnrollmentService> logger) : IEnrollmentService
{
    public Task<EnrollmentResponseDto?> GetByIdAsync(int courseId, int id, Cancellation token ct) =>
        context.Enrollments
            .AsNoTracking()
            .Where(e => e.Id == id && e.CourseId == courseId)
            .Select(e => new EnrollmentResponseDto(e.Id, e.CourseId, e.StudentId, e.EnrolledAt))
            .FirstOrDefaultAsync(ct);

    public async Task<EnrollmentResponseDto> CreateAsync(int courseId, EnrollmentStudentRequest request, Cancellation token ct)
    {
        // TODO 2: Insert a new Enrollment with CourseId = courseId, StudentId = request.StudentId,
        //           and EnrolledAt = DateTime.UtcNow. SaveChangesAsync(ct), log info, then re-read
    }
}

```

```

        //           through GetByIdAsync(courseId, enrollment.Id, ct).
        throw new NotImplementedException();
    }
}

```

Register it scoped in Program.cs next to the course service:

```
builder.Services.AddScoped<IEnrollmentService, EnrollmentService>();
```

Step 4 Build the nested controller

Create Controllers/EnrollmentsController.cs:

```

using Microsoft.AspNetCore.Mvc;
using Tms.Api.Dtos;
using Tms.Api.Services;

namespace Tms.Api.Controllers;

[ApiController]
[Route("api/courses/{courseId:int}/enrollments")]
public class EnrollmentsController(
    ICourseService courseService,
    IEnrollmentService enrollmentService) : ControllerBase
{
    [HttpGet("{id:int}", Name = nameof(GetEnrollment))]
    public async Task<IActionResult> GetEnrollment(int courseId, int id,
        CancellationToken ct)
    {
        var enrollment = await enrollmentService.GetByIdAsync(courseId,
            id, ct);
        return enrollment is not null ? Ok(enrollment) : NotFound();
    }

    [HttpPost]
    public async Task<IActionResult> EnrollStudent(int courseId, Enroll
        StudentRequest request, CancellationToken ct)
    {
        // TODO 3: Look up the parent course (courseService.GetByIdAsyn
        c). If null, return NotFound().
        //           Then check capacity (course.EnrollmentCount >= cours
        e.MaxCapacity).
        //           If full, return Conflict(new ProblemDetails { ... })
        with:
        //           Title = "Course is full"
        //           Detail = $"Course '{course.Title}' has reached its
        maximum capacity of {course.MaxCapacity}."
        //           Status = StatusCodes.Status409Conflict
        //           Otherwise, call enrollmentService.CreateAsync and re
        turn CreatedAtAction(nameof(GetEnrollment),

```

```

        //          new { courseId, id = enrollment.Id }, enrollment).
        throw new NotImplementedException();
    }
}

```

The order matters: **404 before 409**. A client posting into a course that does not exist deserves the 404, not a 409 about a course that was never there.

Step 5 Verify

Step	Action	Expected status	Expected body
1	POST /api/courses with a code from earlier (e.g. CSE-101)	409 Conflict	{ "title": "Course code already exists", "status": 409, "detail": "A course with code 'CSE-101' ..." }
2	Create a fresh course CSE-103 with maxCapacity: 1. Then POST /api/courses/{id}/enrollments with body { "studentId": 1 }	201 Created	Location header points at GetEnrollment; body is the new EnrollmentResponseDto
3	Repeat step 2's POST (so course CSE-103 is now at 1/1)	409 Conflict	{ "title": "Course is full", "status": 409, "detail": "Course '...' has reached its maximum capacity of 1." }
4	POST /api/courses/9999/enrollments body { "studentId": 1 }	404 Not Found	Empty or framework ProblemDetails body

(Step 2 assumes Student with Id = 1 exists from M5's seed data. If it does not, create one in pgAdmin or pick an existing StudentId.)

Troubleshooting

Symptom	Likely cause	Fix
POST with duplicate code returns 500	CodeExistsAsync not called; DB unique constraint hit instead	Add the pre-check in the controller <code>Conflict(new ProblemDetails { ... })</code> before calling <code>CreateAsync</code>
409 body is empty {}	<code>Conflict()</code> called with no argument	Pass new <code>ProblemDetails { Title = "...", Status = 409, Detail = "..." }</code>
Nested route never matches	Route template parameter name and action parameter name disagree	<code>[Route("api/courses/{courseId:int}/enrollments")] ↔ int courseId</code> parameter names must match exactly
Enrolling into a missing course returns 409 instead of 404	Capacity check ran before the parent look-up	Reorder: parent look-up → <code>NotFound()</code> if null → capacity check → <code>Conflict()</code> if full
EnrollmentCount always reads 0 on the parent fetch	<code>CourseService.GetByIdAsync</code> does not project a fresh count	The projection in Exercise 2 reads <code>c.Enrollments.Count</code> from the database every call confirm you re-fetched the course inside the enrolment action, not a stale variable from before the insert

Why this session ends here

You just shipped the contract for one TMS resource end-to-end:

- **Routes name resources, not actions.** A new developer reading `[Route("api/courses")]` knows without any documentation that GET lists, POST creates, and `{id}` is the single-item path. The `{id:int}` constraint stops malformed IDs at the routing layer.
- **The wire and the database are different things.** `CourseResponseDto` is what a client sees; `Course` is what EF persists. A future migration that adds `IsDeleted` or `LastUpdatedBy` does not leak through the API. A migration that renames `Title` does but only because the DTO references `Title` by name, which is exactly the seam where the contract is supposed to break visibly.

- **Validation is declarative and consistent.** Data Annotations + [ApiController] produce 400 ValidationProblemDetails for free. You did not write if (string.IsNullOrEmpty(...)) even once.
- **Business rules return 409, not 500.** A duplicate code or a full course is a known, expected failure; the controller checks the rule, returns Conflict(new ProblemDetails { ... }), and the integration team's HTTP client catches it as an error instead of trusting a 200 and corrupting state.

You did **not** build pagination, links, or rich Scalar metadata yet. That is deliberate. Session 2 takes the same CoursesController and bolts on safe collection retrieval that is where pagination earns its place. Session 3 makes the response self-describing for the Angular team and any integration partner. Today's deliverable is the contract for one resource, lit up across all the failure modes a real client will hit on day one.

Session 1 Checkpoint

Before moving to Session 2, confirm all five rows pass against your live API:

Check	Pass means
POST /api/courses with a valid body	201 Created, Location header points at GET /api/courses/{newId}, body is CourseResponseDto
POST /api/courses with empty {} body	400 Bad Request, body is ValidationProblemDetails with field-level errors
POST /api/courses with a code that already exists	409 Conflict, body is ProblemDetails with Title = "Course code already exists"
POST /api/courses/{id}/enrollments into a course where EnrollmentCount >= MaxCapacity	409 Conflict, body is ProblemDetails with Title = "Course is full"
Any course response inspected in Scalar	No enrollments array, no isDeleted, no other navigation property only the five DTO fields

If any row fails, fix it before Session 2. Pagination assumes the contract for a single resource is solid; you cannot count what you cannot return cleanly.